

PEC 2: Juegos

Presentación

Segunda PEC del curso de Inteligencia Artificial

Competencias

En esta PEC se trabajaran las siguientes competencias:

Competencias de grado:

- Capacidad de analizar un problema con el nivel de abstracción adecuado a cada situación y aplicar las habilidades y conocimientos adquiridos para abordarlo y solucionarlo.

Competencias específicas:

- Saber representar las particularidades de un problema según un modelo de representación del conocimiento.
- Saber resolver problemas intratables a partir de razonamientos aproximados y heurísticos (algoritmos voraces, algoritmos genéticos, lógica difusa, redes bayesianas, redes neuronales, min-max).

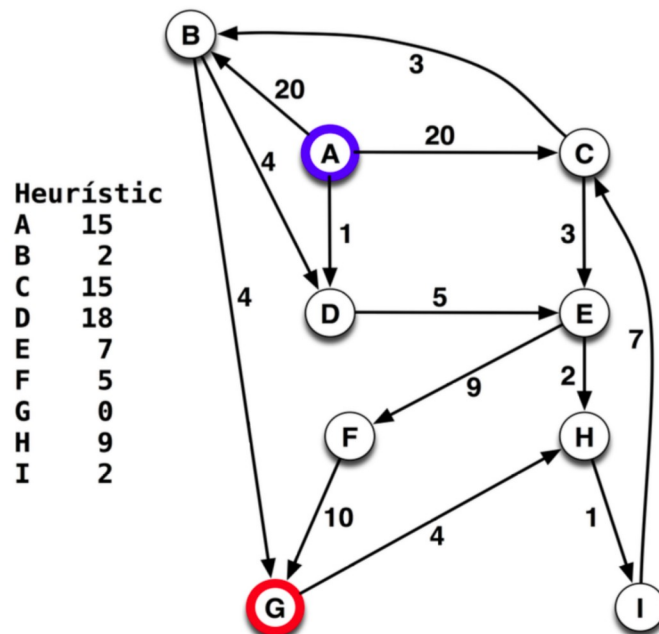
Objetivos

Esta PEC pretende evaluar vuestros conocimientos sobre búsquedas, su formalización y estrategias relacionadas.

Descripción de la PEC/práctica a realizar

El punto de partida de la PEC2 de este semestre es el último apartado (ap. D) de la pregunta 2 de la PEC2 del semestre pasado (directorio 2017_1 set17-feb18 del paquete de PECs de otros semestres que encontraréis en el Aula).

Teníamos el siguiente grafo:



y el programa `astar2.lisp` con el que hacer una búsqueda A* en este grafo. Además, la función heurística no es admisible, tal como podemos ver en la solución de la PEC2.

Se recuerda que en el Módulo 2 hay una implementación en Common Lisp de un sistema de búsqueda general que, con pocas adaptaciones, se puede transformar en diferentes tipos de búsquedas. Así, el código del sistema general está descrito en los apartados 1.1.3, 2.1 y tiene la búsqueda en anchura definida en el apartado 3.1.1, la búsqueda en profundidad en el apartado 3.2, la búsqueda en profundidad limitada en el apartado 3.2.1 y la búsqueda A* a partir del apartado 4.3.2. Para ilustrar el código, éste está aplicado al ejemplo del rompecabezas lineal.

Esperamos que continuéis la reflexión sobre algoritmos de búsqueda, aprovechando este grafo de la PEC2 del semestre pasado.

1.- Sugiere una función heurística **admisible** para este grafo. Justifica como has llegado a encontrar la función.

Recordemos que: *Una función heurística es admisible si nunca sobreestima el coste.* Así pues, hay infinitas funciones heurísticas admisibles. Considerando que todos los costes son positivos, una heurística trivial es $h(n) = 0$ para todo n . Esto convertirá nuestra

búsqueda en una búsqueda de coste uniforme (que también es óptima, como la A* con heurística admisible, pero menos eficiente). Una posible heurística no trivial pero admisible sería:

$$h(A) = 2$$

$$h(B) = 1$$

$$h(C) = 3$$

$$h(D) = 5$$

$$h(E) = 1$$

$$h(F) = 3$$

$$h(G) = 0$$

$$h(H) = 2$$

$$h(I) = 5$$

Se ha encontrado a partir de una inspección de los costes, partiendo de la heurística trivial. Con esta heurística haremos el ejercicio 2. La heurística del problema original era no admisible, basta con mirar $h(C) = 15$ aunque se puede ir de C a G con coste 7.

2.- Realizar varias búsquedas sobre este grafo, a mano (sin ningún *software* auxiliar), utilizando la función heurística que se ha propuesto en la pregunta 1 (no la que hay listada junto al grafo):

2.1.- Búsqueda de coste uniforme

Inicio:

Open: (A 0)

Closed:

Iteración 1: Open: (D 1) (C 20) (B 20)

Closed: (A 0)

Iteración 2: Open: (E 6) (B 20) (C 20)

Closed: (D 1) (A 0)

Iteración 3: Open: (H 8) (F 15) (C 20) (B 20)

Closed: (E 6) (D 1) (A 0)

Iteración 4: Open: (I 9) (F 15) (B 20) (C 20)

Closed: (H 8) (E 6) (D 1) (A 0)

Iteración 5: Open: (F 15) (C 16) (C 20) (B 20)

Closed: (I 9) (H 8) (E 6) (D 1) (A 0)

Iteración 6: Open: (C 16) (B 20) (C 20) (G 25)

Closed: (F 15) (I 9) (H 8) (E 6) (D 1) (A 0)

Iteración 7: Open: (E 19) (B 19) (C 20) (B 20) (G 25)

Closed: (C 16) (F 15) (I 9) (H 8) (E 6) (D 1) (A 0)

Iteración 8: Open: (B 19) (B 20) (C 20) (H 21) (G 25) (F 28)

Closed: (E 19) (C 16) (F 15) (I 9) (H 8) (E 6) (D 1) (A 0)

Iteración 9: Open: (C 20) (B 20) (H 21) (G 23) (D 23) (G 25) (F 28)

Closed: (B 19) (E 19) (C 16) (F 15) (I 9) (H 8) (E 6) (D 1) (A 0)

Iteración 10: Open: (B 20) (H 21) (D 23) (G 23) (E 23) (B 23) (G 25) (F 28)

Closed: (C 20) (B 19) (E 19) (C 16) (F 15) (I 9) (H 8) (E 6) (D 1) (A 0)

Iteración 11: Open: (H 21) (B 23) (E 23) (G 23) (D 23) (G 24) (D 24) (G 25) (F 28)

Closed: (B 20) (C 20) (B 19) (E 19) (C 16) (F 15) (I 9) (H 8) (E 6) (D 1) (A 0)

Iteración 12: Open: (I 22) (D 23) (G 23) (E 23) (B 23) (D 24) (G 24) (G 25) (F 28)

Closed: (H 21) (B 20) (C 20) (B 19) (E 19) (C 16) (F 15) (I 9) (H 8) (E 6) (D 1) (A 0)

Iteración 13: Open: (B 23) (E 23) (G 23) (D 23) (G 24) (D 24) (G 25) (F 28) (C 29)

Closed: (I 22) (H 21) (B 20) (C 20) (B 19) (E 19) (C 16) (F 15) (I 9) (H 8) (E 6) (D 1) (A 0)

Iteración 14: Open: (D 23) (G 23) (E 23) (D 24) (G 24) (G 25) (G 27) (D 27) (F 28) (C 29)

Closed: (B 23) (I 22) (H 21) (B 20) (C 20) (B 19) (E 19) (C 16) (F 15) (I 9) (H 8) (E 6) (D 1) (A 0)

Iteración 15: Open: (E 23) (G 23) (G 24) (D 24) (G 25) (D 27) (G 27) (F 28) (E 28) (C 29)

Closed: (D 23) (B 23) (I 22) (H 21) (B 20) (C 20) (B 19) (E 19) (C 16) (F 15) (I 9) (H 8) (E 6) (D 1) (A 0)

Iteración 16: Open: (G 23) (D 24) (G 24) (G 25) (H 25) (G 27) (D 27) (E 28) (F 28) (C 29) (F 32)

Closed: (E 23) (D 23) (B 23) (I 22) (H 21) (B 20) (C 20) (B 19)
(E 19) (C 16) (F 15) (I 9) (H 8) (E 6) (D 1) (A 0)

Fijémonos en el resultado (el camino óptimo) y en las iteraciones que ha tardado en conseguirlo. Compárese con la búsqueda A*

2.2.- Búsqueda ávida

Inicio

Open: (A 2)

Closed: -

Iteración 1: Open: (B 1) (C 3) (D 5)

Closed: (A 2)

Iteración 2: Open: (G 0) (C 3) (D 5) (D 5)

Closed: (B 1) (A 2)

Itera solo DOS veces, pero NO llega a la solución óptima (esta solución, A-B-G tiene coste 24).

2.3.- Búsqueda A*: En este caso compárese el resultado con el que se obtiene utilizando la función heurística no admisible que hay listada junto al dibujo del grafo

Inicio

Open: (A 2)

Closed: -

Iteración 1: Open: (D 6) (B 21) (C 23)

Closed: (A 2)

Iteración 2: Open: (E 7) (B 21) (C 23)

Closed: (D 6) (A 2)

Iteración 3: Open: (H 10) (F 18) (B 21) (C 23)

Closed: (E 7) (D 6) (A 2)

Iteración 4: Open: (I 14) (F 18) (B 21) (C 23)

Closed: (H 10) (E 7) (D 6) (A 2)

Iteración 5: Open: (F 18) (C 19) (B 21) (C 23)

Closed: (I 14) (H 10) (E 7) (D 6) (A 2)

Iteración 6: Open: (C 19) (B 21) (C 23) (G 25)

Closed: (F 18) (I 14) (H 10) (E 7) (D 6) (A 2)

Iteración 7: Open: (E 20) (B 20) (B 21) (C 23) (G 25)

Closed: (C 19) (F 18) (I 14) (H 10) (E 7) (D 6) (A 2)

Iteración 8: Open: (B 20) (B 21) (C 23) (H 23) (G 25) (F 31)

Closed: (E 20) (C 19) (F 18) (I 14) (H 10) (E 7) (D 6) (A 2)

Iteración 9: Open: (B 21) (H 23) (C 23) (G 23) (G 25) (D 28) (F 31)

Closed: (B 20) (E 20) (C 19) (F 18) (I 14) (H 10) (E 7) (D 6) (A 2)

Iteración 10: Open: (G 23) (C 23) (H 23) (G 24) (G 25) (D 28) (D 29) (F 31)

Closed: (B 21) (B 20) (E 20) (C 19) (F 18) (I 14) (H 10) (E 7) (D 6) (A 2)

Esta solución es mejor que la obtenida con la función heurística no admisible, es decir, el coste es inferior.

3.- Modificar el programa `astar2.lisp` para poder realizar búsquedas de coste uniforme y búsquedas ávidas (además de la búsqueda A*, que es lo que hace).

Ver el archivo `busquedas-pac2.lisp`, adjunto con la solución.

Esencialmente los cambios son muy pocos, ya que:

a) Coste Uniforme - Búsqueda A* con $h(\text{nodo}) = 0$ para todo nodo:

- Función `tl-estrategia-coste-uniforme`: la misma que `tl-estrategia-A*`

- Fijémonos que la función `problema-busqueda-coste-uniforme` es la misma que `problema-busqueda-A*` donde cada llamada a `(heurística estado)` ha sido sustituida por 0.

- Finalmente,

```
(defun busqueda-coste-uniforme (problema)
```

```
  (hacer-busqueda problema #'tl-estrategia-coste-uniforme))
```

b) Ávida - Búsqueda A* sin considerar el coste, es decir, $g(\text{nodo}) = 0$ para todo nodo:

- Función `tl-estrategia-avida`: la misma que `tl-estrategia-A*`

- Fijémonos que la función **problema-busqueda-avida** es la misma que **problema-busqueda-A*** donde cualquier referencia al coste $g(\text{nodo})$ ha sido eliminada y sustituida por 0.

- Finalmente,

```
(defun busqueda-avida (problema)
  (hacer-busqueda problema #'tl-estrategia-avida))
```

Recursos

Para realizar esta PEC el material imprescindible son los temas 2, 3 y 4 del Módulo 2.

Criterios de valoración

Problema 1: 10%

Problema 2.1: 20%

Problema 2.2: 20%

Problema 2.3: 20%

Problema 3: 30%

Formato y fecha de envío

Para dudas y aclaraciones sobre el enunciado, dirigiros al consultor responsable del aula.

Hay que entregar la solución en un archivo **PDF**. Adjuntar el fichero a un mensaje en el apartado Entrega y Registro de EC (REC).

El nombre del archivo debe ser Apellidos_Nombre_IA_PEC2 con la extensión. pdf (PDF).

La fecha límite de entrega es el: **13 de Abril** (a las 24 horas, más o menos).

Razonad la respuesta en todos los ejercicios. Las respuestas sin justificación no recibirán puntuación.

Nota: Propiedad intelectual

A menudo es inevitable, al producir una obra multimedia, hacer uso de recursos creados por terceras personas. Es por tanto comprensible hacerlo en el marco de una práctica de los estudios de Informática, siempre que se documente claramente y no suponga plagio en la práctica.

Por lo tanto, al presentar una práctica que haga uso de recursos ajenos, se presentará junto con ella un documento en el que se detallen todos ellos, especificando el nombre de cada recurso, su autor, el lugar donde se obtuvo y el su estatus legal: si la obra está protegida por copyright o se acoge a alguna otra licencia de uso (Creative Commons, licencia GNU, GPL ...). El estudiante deberá asegurarse de que la licencia que sea no impide específicamente su uso en el marco de la práctica. En caso de no encontrar la información correspondiente deberá asumir que la obra está protegida por copyright. Deberán, además, adjuntar los archivos originales cuando las obras utilizadas sean digitales, y su código fuente esté correspondiente.